QA evidence — stress + chaos (§11.4.85)

In-process black-box resilience suite over the real `Server.Router()` (`server/internal/api/resilience_test.go`), run under `go test -race`.

- **Stress** — concurrent group-create (200 parallel, 0 errors), sustained reads (2400 requests across 16 workers, 0 errors), concurrent membership contention (60 distinct devices added to one group in parallel → exactly 60, no lost updates).
- **Chaos** — injected repository fault (`faultRepo`) → `GET /groups` degrades to 500 gracefully (no panic), stays 500 under sustained fault, then **recovers** to 200 when the fault clears.

`run.log` holds the captured per-test latency (p50/p95/p99) + error census + the chaos recovery transition. `-race` clean ⇒ the concurrent paths are data-race-free. Reproduce: `cd server && go test -race ./internal/api/ -run 'TestStress|TestChaos' -v`.

DDoS / flood (§11.4.27 ddos type) — added 2026-06-08

`TestDDoSFloodStaysUpAndRecovers`: a 5,952-request burst across 64 workers at `/healthz`; the server serves every request without panic/hang and is immediately responsive afterward (post-flood `/healthz`→200 and an authed `POST /groups`→201). Run under `-race` (clean).

FINDING (honest, not a fake pass): the MVP control plane has NO rate-limiting / concurrency cap — 0 of 5,952 requests were shed (no 429s); the only protection under flood is the host + Go scheduler. **RECOMMENDATION (pre-production):** add a rate-limit / concurrency-cap middleware (e.g. token-bucket per client/IP, or a bounded in-flight semaphore) before any public exposure. Tracked here as a security-hardening gap; the test verifies graceful-under-flood + recovery, and will assert load-shedding (429s) once a limiter lands.

UPDATE — recommendation IMPLEMENTED (2026-06-08)

The "no rate-limiting" finding above is now addressed: an **in-flight cap middleware** (`maxInflightMiddleware`, env `HELIX_MAX_INFLIGHT`, default 0=off) sheds excess concurrency with **429 RATE_LIMITED** + `Retry-After`. Proven by `TestMaxInflightShedsUnderFlood` (cap=1 under 300 concurrent → 244 shed / 56 served / responsive post-flood, race-clean) and `TestMaxInflightDisabledByDefault` (0 sheds when off — existing behaviour

preserved). Operative recommendation now: **set `HELIX_MAX_INFLIGHT` to a host-appropriate bound before public exposure** (a per-IP token-bucket remains a future refinement on top of the global cap).